

Mini-Project (ML for Time Series) - MVA 2024/2025

Jules Chapon jules.chapon@ensae.fr
Corentin Pernot corentin.pernot@ensae.fr

January 8, 2025

1 Introduction and contributions

Time series classification aims to assign labels to time series based on their characteristics. This task is challenging due to the variety and complexity of patterns in the data. One idea to tackle this problem is to identify specific patterns that are discriminative for each class and can help classify new, unlabeled time series.

A concept called Shapelets addresses this challenge. Shapelets, introduced in [1], are subsequences of time series that are highly representative of a particular class. They extend the nearest neighbor algorithm and have shown good results in classification tasks. One advantage of Shapelets is their interpretability. Since they are subsequences that clearly differentiate between classes, they provide meaningful insights into the data. This makes them particularly useful in applications like gesture recognition or healthcare, where understanding the decision-making process is essential. However, discovering Shapelets is computationally demanding. The brute-force algorithm required to find them has a high computational cost, making it impractical for large datasets.

The paper under study addresses this issue by proposing a new method for computing Shapelets more efficiently, leveraging the SAX representation while achieving the same results as the brute-force algorithm. The implementation of this Fast Shapelets Algorithm is available in C++, but we chose to implement it from scratch in Python to better understand the subtleties and design choices made in the original paper. However, we utilized existing Python functions, such as entropy and Dynamic Time Warping (DTW) distance. We applied Fast Shapelets to the ECGFive-Days dataset to evaluate its performances in terms of accuracy and runtime. We compared these performances with a 1-Nearest-Neighbor classifier with both euclidean and DTW distance. We conducted experiments with the parameters used in the paper for another dataset such as the number of iterations and the number of retrieved candidates. We tried to challenge some choices by testing different configurations, including cardinality and dimensionality. Additionally, we assessed the robustness of the method to noise to better understand how it performs on more realistic, noisy data. While we both implemented the fast shapelet algorithm to gain a thorough understanding of its steps, challenges, and to collaborate on improvements, Corentin Pernot concentrated primarily on optimizing the code (through vectorization and parallelization), whereas Jules Chapon focused on setting up and conducting the experiments.

2 Method

The paper proposes a novel method for computing Shapelets by leveraging approximations through SAX (Symbolic Aggregate approXimation) representations. Considering a dataset with n time series and m as the length of the longest series, the brute-force algorithm for computing distances between a candidate and all time series has a complexity of $\mathcal{O}(n^2m^4)$. In contrast, the proposed Fast Shapelet algorithm achieves a reduced complexity of $\mathcal{O}(nm^2)$. The overall process is divided into two distinct phases.

- In the first phase, the objective is to select subsequence candidates that demonstrate high discriminative power in the SAX representation. Given a new dimensionality $d = 16$ and an alphabet cardinality $c = 4$, the SAX representation reduces the dimensionality of the data while discretizing them. For each time series, given a predefined word length, the sliding window technique described in [2] is applied to compute the SAX representation. Since this technique uses a step size of one, some consecutive words are identical. As suggested in [2], we chose to exclude these redundant words. Consequently, each time series is represented by a set of distinct SAX words, where the number of words may vary between series, but their length remains fixed.

To address the issue where minor variations in the time series can produce different SAX representations, the method introduces a random masking step. This process transforms, for instance, **acdba** and **bcdca** into the same representation, ***cd*a**. While this reduces the number of false negatives among candidates, excessive masking can increase the number of false positives. To balance this trade-off, we decided that the event for a letter to be masked follows a Bernoulli distribution of parameter 0.2.

For each random masking iteration, a collision table is computed. The authors argue that objects which are similar in the original space are likely to collide in the SAX representation, thus facilitating their identification.

After $r = 10$ iterations of random masking, the collision table is expected to remain largely sparse, with a few locations containing values that are close to r . The next step involves calculating the *distinguishing power* of each SAX word by analyzing the collision table and aggregating object-based counts with respect to their class. The distinguishing power of a SAX word is strongly correlated with the quality of the corresponding shapelet.

At the end of this first phase, the top K candidates in the SAX representations, those with the highest distinguishing power, are selected. We set $K = 10$ as in the original paper. These SAX words are then mapped back to their original time series representation.

This approach significantly reduces the computational cost by avoiding all-to-all distance calculations across all subsequences of all time series in their high-dimensional space. Instead, it computes collisions in the low-dimensional SAX representation space, which is more efficient computationally and faster for identifying candidate shapelets. The all-to-all distance computations are then performed only on the top K retrieved candidates. While we will analyze the influence of this parameter later, it is crucial to ensure that the candidates identified in the SAX representation space are also highly discriminative in the original time series space.

- The second phase of the algorithm involves testing these candidates and selecting the most discriminative one regarding the information gain criterion. To achieve this, we first need to determine the best split ($\langle \text{candidate}, \text{threshold} \rangle$) for each candidate, which is defined as the split that maximizes the information gain. Information gain measures the reduction in uncertainty (entropy) about the target variable after splitting the dataset based on the minimal distance between subsequences and a given shapelet. Then, among all the shapelet candidates and their corresponding thresholds, we select the one with the highest information gain, breaking ties by the maximum separation gap. The separation gap is defined as the difference between the average minimal distance to the shapelet for one class and the average minimal distance for the other class.

The procedure outputs a dictionary that maps each tested length to its corresponding best shapelet. Since the computations for different lengths are independent, we parallelized the process using `joblib`.

The implementation can be used in two ways. The first approach is applicable when the user knows the desired length of the shapelet or pattern. In this case, the implementation returns a single shapelet of the specified length. We can then compute the minimal distance between each time series and this shapelet, and classify new time series in the test set using the threshold learned from the training set.

The second approach, which is more reflective of real-world *scenarii*, assumes that the length of the final shapelet is unknown. In this case, the user specifies a range by setting minimum and maximum lengths, and the implementation identifies the best shapelet for each length within this range. The idea is to transform the original dataset by creating features that represent, for each time series, the minimal distance to the corresponding shapelet. These features can then be used with a classifier that maximizes the information gain—such as a decision tree—to classify new time series, as described in [1]. Additionally, this method provides insights into which shapelet and length are the most discriminative.

3 Data

We chose to use the ECGFiveDays dataset available on <https://www.timeseriesclassification.com> and provided by Y. Chen and E. Keogh. The ECGFiveDays dataset consists of univariate time-series data representing electrocardiogram (ECG) recordings, which measure the heart’s electrical activity over time from the same patient. Each series contains 136 values sampled at regular intervals, capturing variations in the cardiac signal. The dataset is divided into two classes: one representing normal ECG readings and the other containing abnormal signals, often indicative of potential cardiac anomalies. Classes are present in similar proportions in both train and test data. The data is preprocessed and normalized and not very noisy, making it suitable for immediate use in tasks such as anomaly detection and biomedical signal classification. With a small training set (23 instances, with a 60%-40% ratio between classes) and a larger test set (861 instances, with a 50%-50% ratio between classes), the dataset highlights the variability in ECG signals and commonly serves as a benchmark for evaluating classification models and pattern recognition techniques, as demonstrated in the last section of [1]. As noted in this paper, one of the main challenges in classifying this dataset is the presence of a gradual trend (referred to as a wandering baseline in medical contexts) and the fact that time series from the two classes are globally very similar (see figure 1).

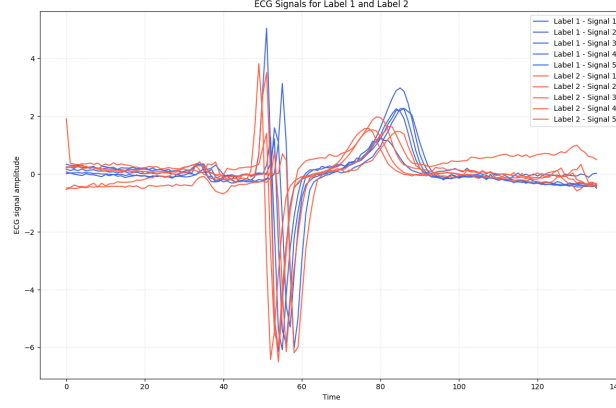


Figure 1: 5 signals of each class

4 Results

We first analyze the binary classification task. To achieve classification, we use the second approach described earlier. The length of the most discriminant shapelet was assumed to be unknown. Thus, we compute the best shapelet for each length between 1 and 136. We then select the split $\langle \text{shapelet}, \text{threshold} \rangle$ that maximizes the information gain (decision tree with 'max_length' = 1). As the algorithm is random (due to the random projection), the length of the selected shapelet can vary (32, 39, 42, 78...), and so its accuracy (see figure 4). The best performance is achieved when the model returns a shapelet of size 42 : it has an accuracy of 100% on the training set, and of 99.5% on the test set -the same result that they obtained in the original paper.

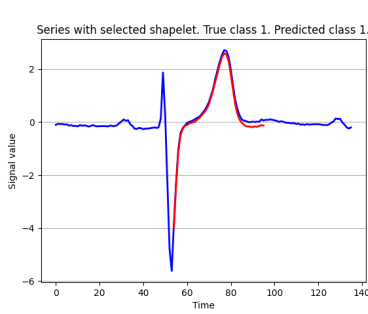


Figure 2: Most discriminant Shapelet (of length 42) and a signal for this class

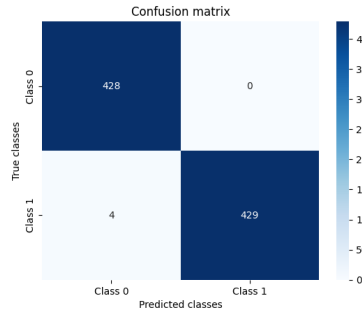


Figure 3: Confusion matrix on the test set associated to this shapelet of size 42

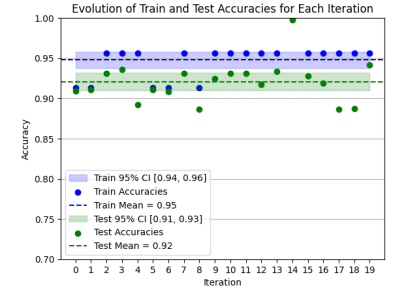


Figure 4: Randomness: Accuracies over 20 different iterations

We compared these results with those achieved by the brute-force shapelet algorithm. This brute-force algorithm helps us compute the optimal shapelet for a given length in 30 seconds, while the Fast Shapelet algorithm can compute each shapelet of length between 1 and 136 in less than 15 seconds. On this dataset, for a given shapelet length, the two algorithms achieve the same performance.

We also compared Fast Shapelet with the 1-Nearest-Neighbor classifier using both Euclidean distance and Dynamic Time Warping (DTW) as distance metrics. Using Euclidean distance, we

achieve 80% accuracy on the test set rapidly, while using DTW, we obtain the same performance (79%) but after running for almost 10 minutes.

To this end, Fast Shapelet is a good algorithm to obtain similar results to the brute-force algorithm while being much faster on this dataset.

After this, we decided to test the effect of all parameters on both the accuracy and the training time, as they did in the original paper for K and r .

Regarding these two parameters, we get the same results. Increasing K or r does not increase the accuracy, but it slightly increases the training time (see figures 5, 15, 7, 8).

But to go further than the original paper, we also tested the effect of the other parameters that were not mentionned : the dimensionality, the cardinality and the probability of the mask. As expected, the cardinality does not play a major role as long as it is not too low (1 or 2 for example). The probability of the mask does not affect the training time, and setting it to 0.5 increases the variance in the results. The only parameter that seems to play a major role is the dimensionality, as it increases the training time and the variance in the results (see figures 9, 10, 11, 12, 13, 14).

Finally, a key point is to know whether or not the algorithm is well-suited for real-life *scenarii*, i.e. when there is noise in the data. To do so, we added noise to the data (only on the test set, or on both the training and the test sets), and compared the accuracy of the Fast Shapelets algorithm to a 1-Nearest-Neighbor. And even though Fast Shapelets had better results without noise, it gets outperformed by the 1-Nearest-Neighbor as we add noise (see figures 6, 16).

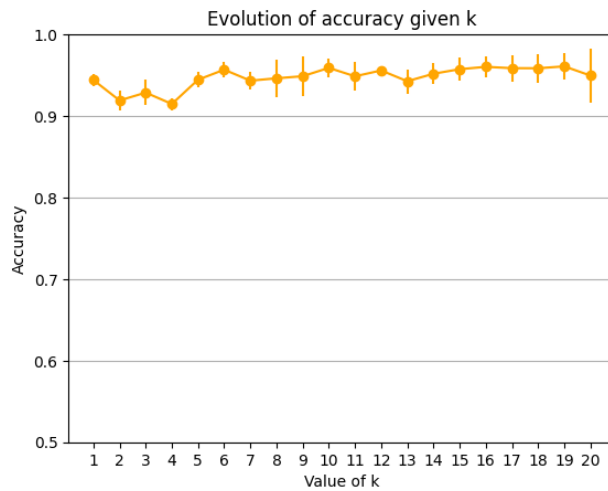


Figure 5: Effect of K on the accuracy

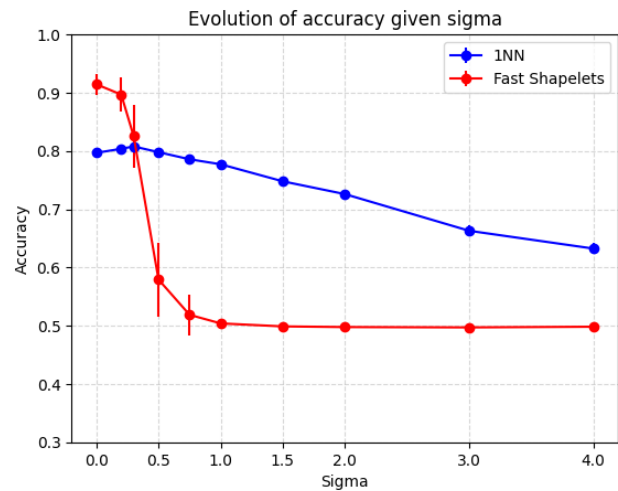


Figure 6: Shapelets vs. 1NN on noisy test data

References

- [1] Thanawin Rakthanmanon and Eamonn Keogh. Fast shapelets: A scalable algorithm for discovering time series shapelets. *Proceedings of the 2013 SIAM International Conference on Data Mining*, 2013. [pdf](#).
- [2] L. Wei J. Lin, E. Keogh and S. Lonardi. Experiencing sax: a novel symbolic representation of time series. *DMKD*, 2007. [pdf](#).

A Additional Graphs

A.1 Parameters Effects

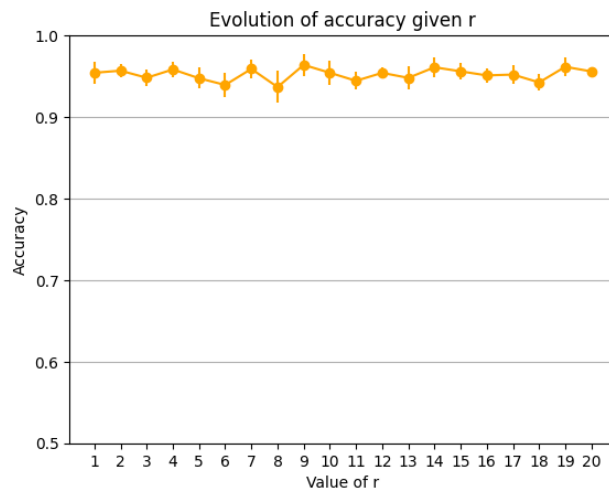


Figure 7: Effect of r on accuracy

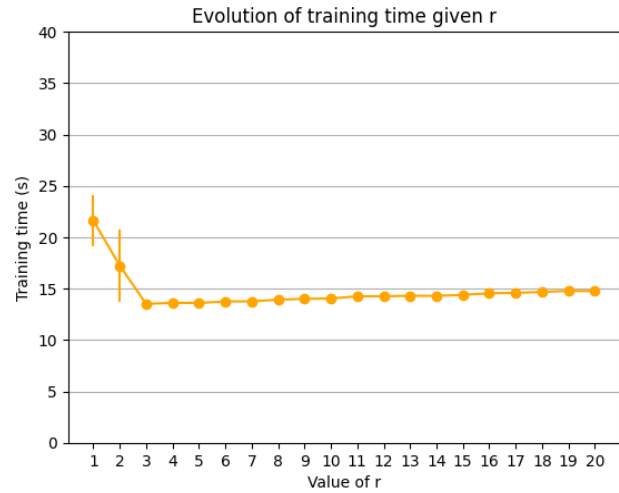


Figure 8: Effect of r on training time

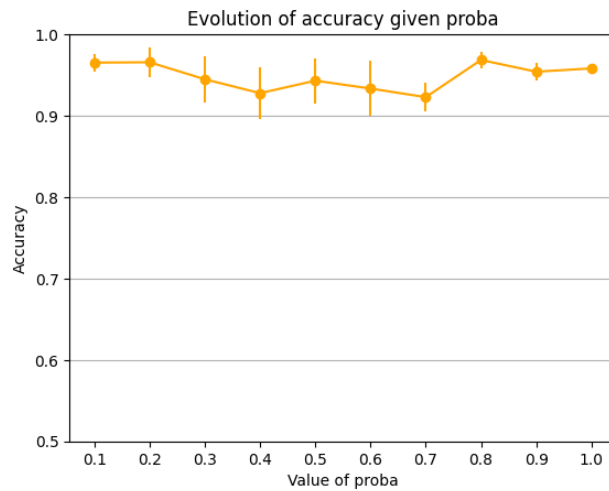


Figure 9: Effect of p on accuracy

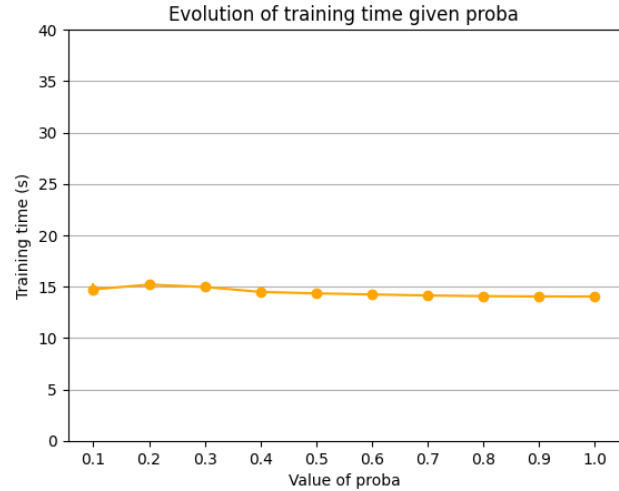


Figure 10: Effect of p on training time

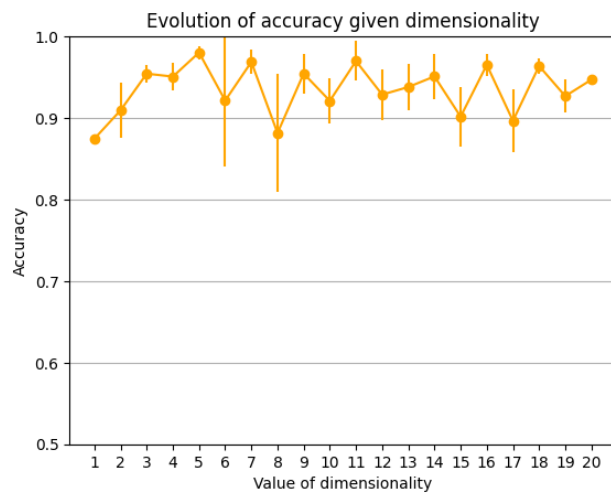


Figure 11: Effect of d on accuracy

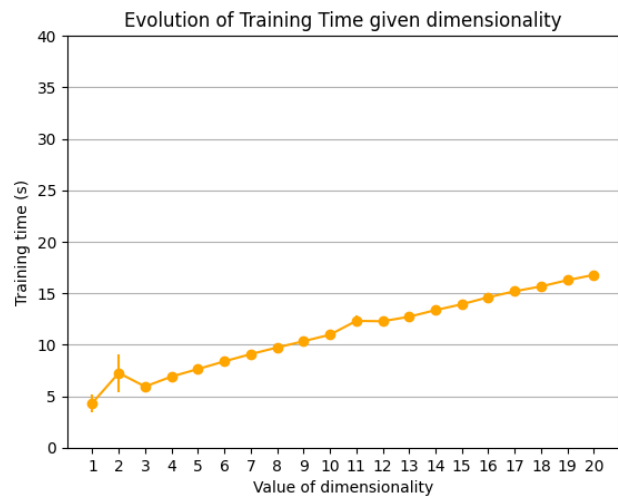


Figure 12: Effect of d on training time

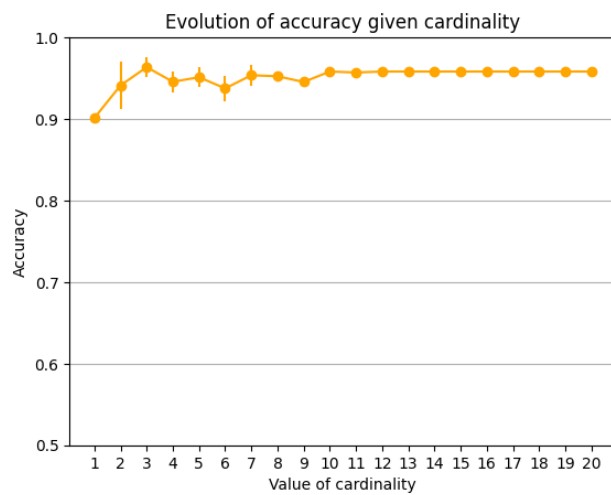


Figure 13: Effect of c on accuracy

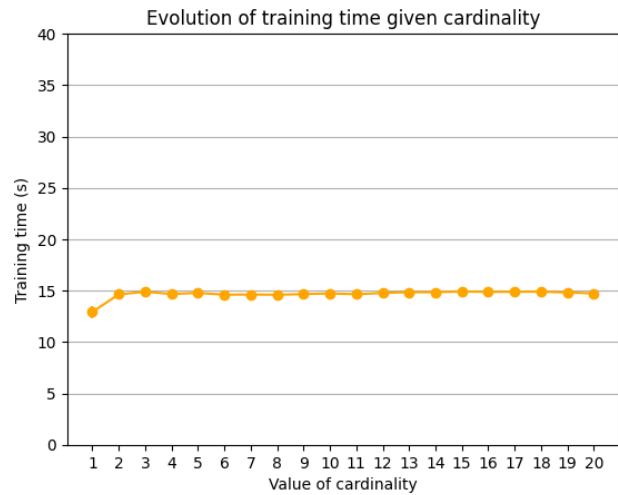


Figure 14: Effect of c on training time

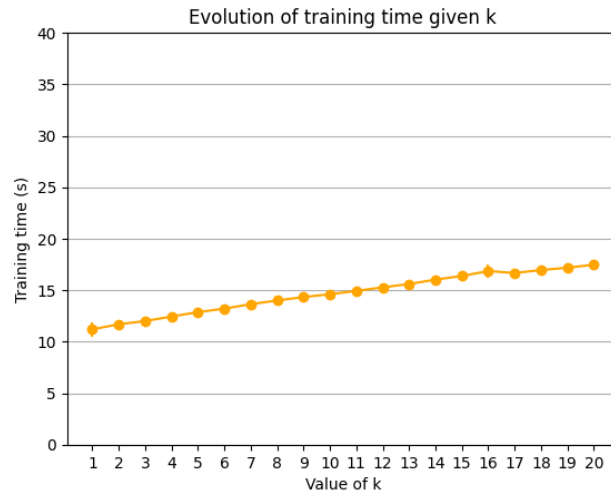


Figure 15: Effect of K on training time

A.2 Noise in data

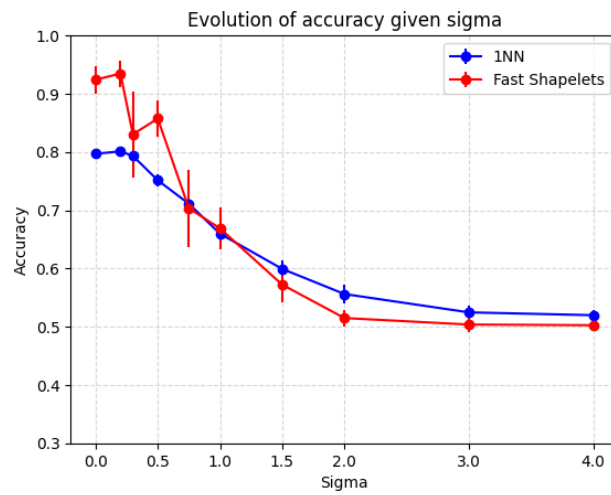


Figure 16: Shapelets vs. 1NN on noisy train and test data